

Documentation Technique

Réflexions Initiale Technologique

Pour mes premières considérations relatives à la pile technologique, j'ai d'abord dressé une liste de quelques technologies pour chaque catégorie, puis j'ai pris en compte les avantages de chacune d'elles, ce qui m'a permis de choisir la pile technologique finale du projet.

Considérations initiales:

- Interface utilisateur: React, Astro
- Domaine serveur: Node, Laravel
- Base de données SQL: MySQL, MariaDB
- Base de données NoSQL: MongoDB
- Déploiement: Digital Ocean

J'ai finalement opté pour cette pile technologique:

- Interface utilisateur: React
- Base de données: Laravel
- Base de données SQL: MySQL
- Base de données NoSQL: MongoDB
- Déploiement: Digital Ocean

J'ai choisi React plutôt qu'un framework comme Astro, car je connais mieux React personnellement et React est un framework frontend standard de l'industrie, ce qui signifie qu'il bénéficie d'un support de longue date pour son développement, sa documentation et qu'il existe de nombreuses entreprises et développeurs qui l'utilisent, ce qui faciliterait la résolution des problèmes bloquants.

Pour le backend, j'ai l'habitude d'utiliser Node.js avec JavaScript et des frameworks comme Express ou Fastify pour créer une API. Cependant, j'ai opté pour Laravel pour plusieurs raisons. La première, et sans doute la plus importante, est la sécurité: Laravel intègre l'authentification des utilisateurs et des sessions, ce qui garantit une sécurité optimale, contrairement à ce qui se passe avec Node.js. De plus, Laravel est un framework PHP riche en fonctionnalités et figure parmi les plus populaires pour le développement backend PHP.

MySQL est préférable à MariaDB, notamment pour sa popularité, et MongoDB est une base de données NoSQL. De manière générale, plus les technologies sont populaires et largement utilisées, mieux c'est: elles offrent des fonctionnalités plus robustes, moins de bugs et de problèmes de sécurité, et bénéficient globalement d'un meilleur support.

J'ai finalement opté pour la containerization avec Docker afin de séparer mon environnement de développement local et de le recréer facilement. Ainsi, toute personne travaillant sur ce projet peut le faire exactement de la même manière. J'ai également utilisé Docker pour déployer le projet sur Digital Ocean, un service d'hébergement VPS très populaire.

Configuration de l'environnement local

J'utilise un MacBook comme environnement local, et j'espère que la majeure partie de la pile technologique était déjà configurée dans mon environnement local grâce aux développements précédents.

En choisissant d'utiliser Docker pour séparer l'environnement en conteneurs et ainsi garantir un environnement de développement local identique pour tous, je n'aurais pas besoin de beaucoup de configuration. Il me suffirait d'installer et de configurer:

- Nodejs/npm
- Git
- IDE (Integrated Development Environment)
- Php
- Composer
- Docker Desktop

Personnellement, je préfère une configuration légère et épurée ; j'évite donc généralement d'installer des paquets ou des applications inutiles. Avant de me lancer dans ce projet, j'ai donc créé un petit projet personnel appelé dotscript (combinaison de fichiers de configuration et de scripts). Ce projet est un ensemble de scripts Bash et de fichiers d'environnement qui me permettent de configurer mon environnement local pour le développement de mes projets habituels. Il s'agit d'un ensemble de scripts qui automatisent l'installation de mon environnement de développement local à l'aide de Bash et de Homebrew (un gestionnaire de paquets populaire sur macOS).

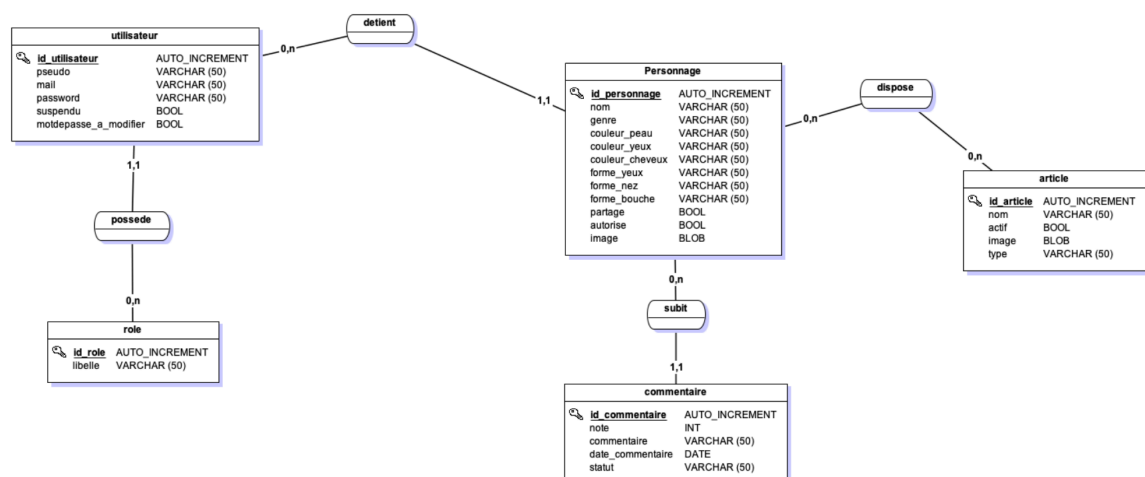
J'ai également automatisé l'installation de mes applications, comme iTerm2 (un terminal populaire pour Mac), dont j'ai absolument besoin puisque j'utilise Neovim qui s'exécute dans le terminal et que je configure avec mes fichiers de configuration Neovim dans ce projet, que je lie symboliquement au bon chemin afin de pouvoir le personnaliser au point d'en faire mon propre IDE personnel.

Grâce à ces scripts, je peux automatiser la préparation fastidieuse de mon environnement local de développement. Sans eux, je devrais tout configurer manuellement: installer Homebrew et tous les paquets, toutes les applications et copier tous mes fichiers de configuration depuis un dépôt Git, par exemple.

Database Entity Relational Diagram (ERD)

J'ai utilisé le modèle MCD fourni pour le transformer en un Entity Relational Diagram (ERD) montrant comment la base de données doit être structurée.

Annexe 1 : schémas de base de données (MCD)



J'ai utilisé des clés étrangères pour les relations One-to-Many, comme entre utilisateurs et personnages, car un utilisateur peut avoir créé plusieurs personnages, mais un personnage ne peut être créé que par un seul utilisateur. La clé étrangère pointe donc vers la table des personnages. Pour les relations plus complexes, telles que les rôles, les commentaires et les éléments, qui présentent une relation Many-to-Many avec une autre table, j'ai créé une table associative ou pivot, comme dans l'exemple des commentaires et des notes. En effet, un commentaire peut avoir plusieurs notes, et une note utilisateur peut être attribuée à plusieurs commentaires, de la même manière qu'entre les personnages et les éléments.

J'ai créé mes fichiers de migration à partir de ces diagrammes, et j'ai également créé des fichiers SQL dans le répertoire doc du projet pour montrer l'équivalent.

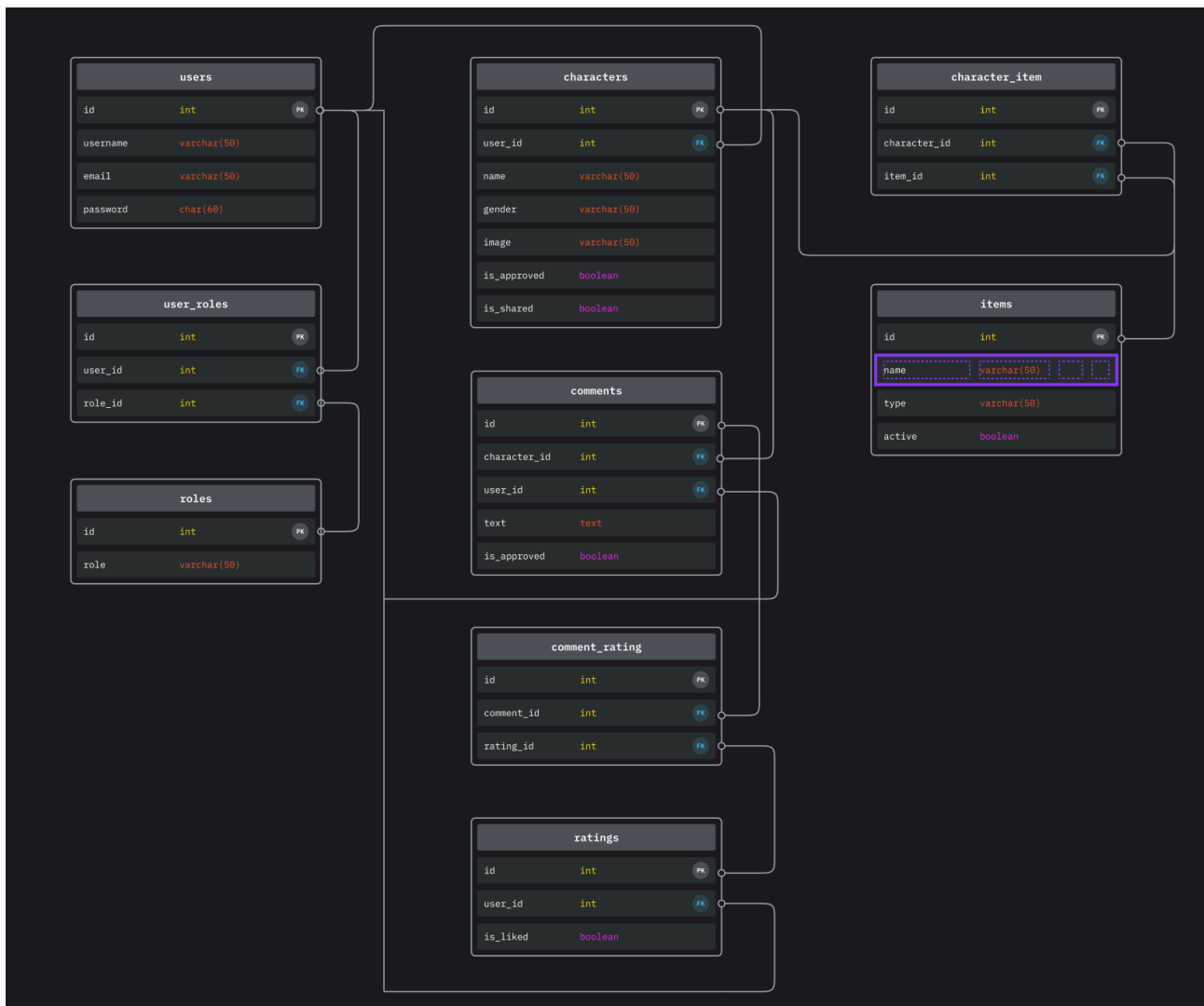


Diagramme d'utilisation et diagramme de séquence

Diagramme d'utilisation:

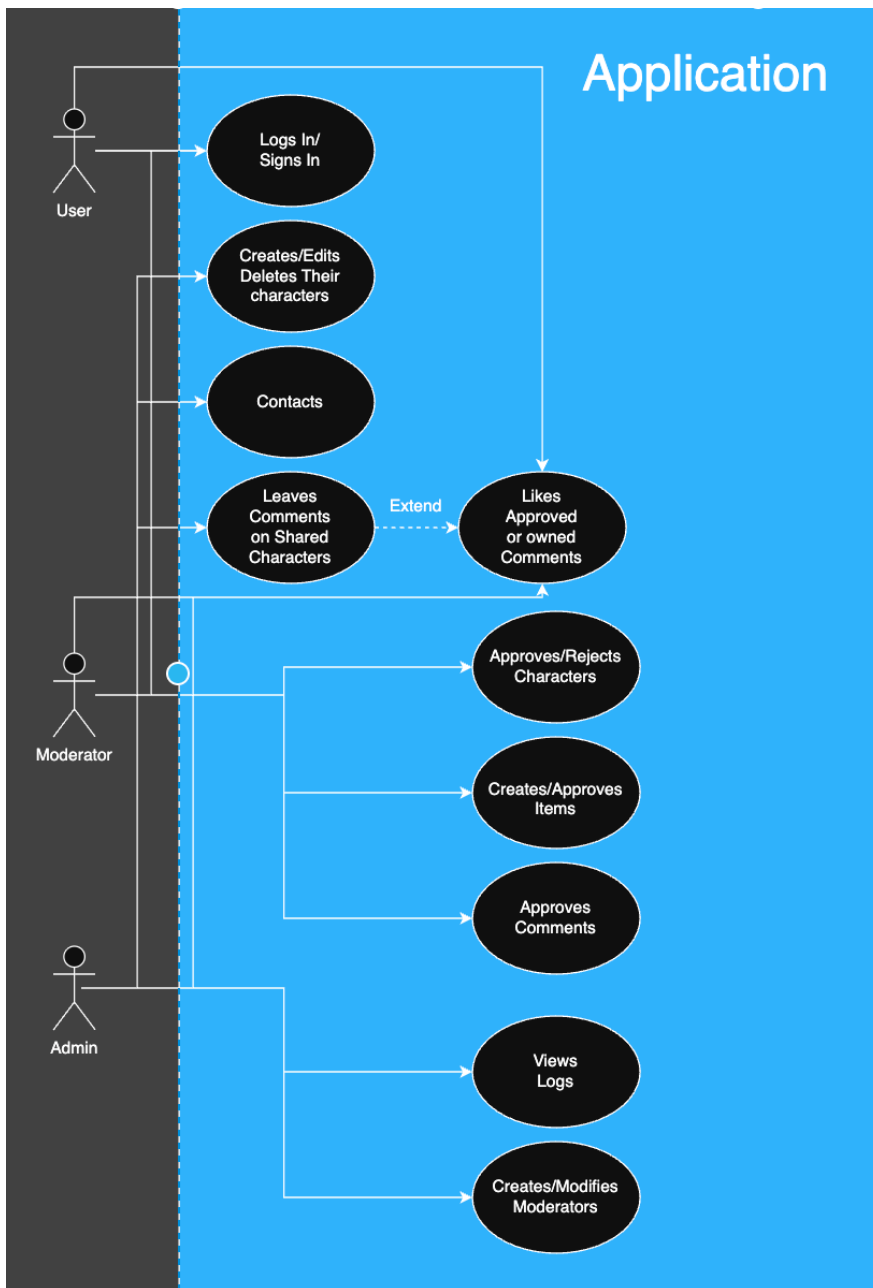
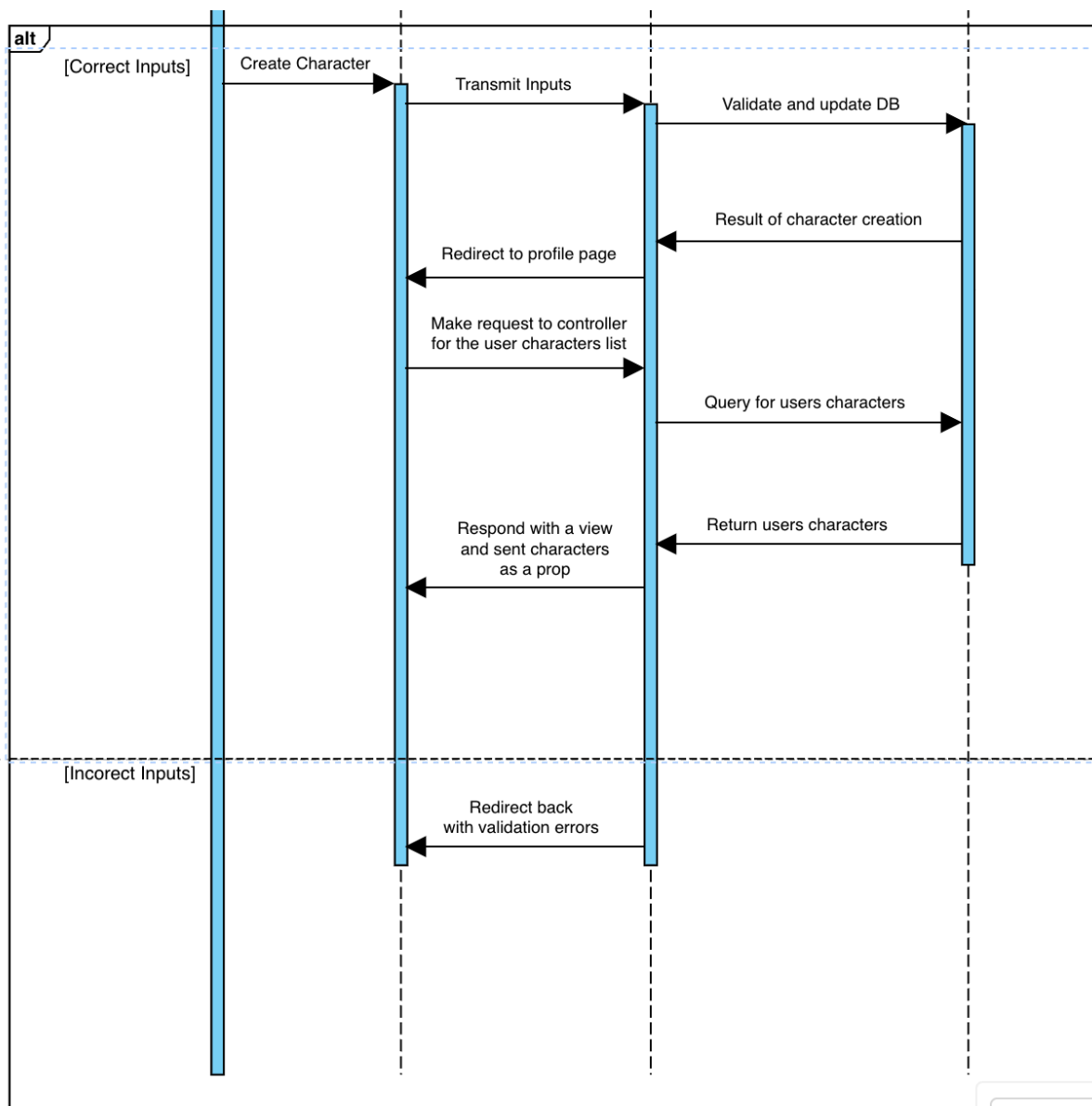
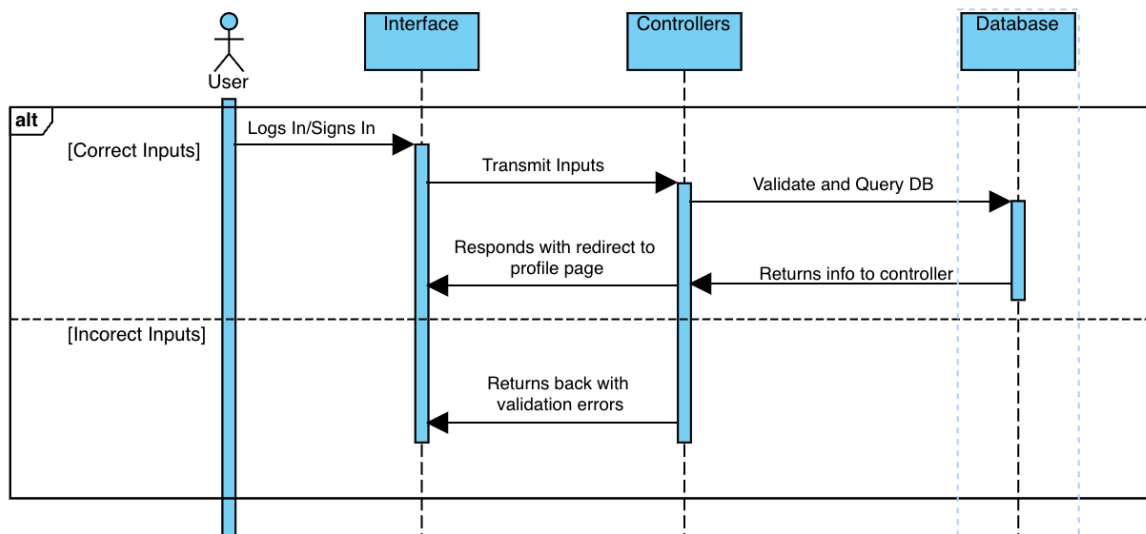


Diagramme de séquence:



Deux diagrammes très utiles pour ce projet sont le diagramme de cas d'utilisation et le diagramme de séquence, comme illustré ci-dessus.

Le diagramme de cas d'utilisation est le premier à créer et montre les utilisateurs de l'application Web et ce qu'ils peuvent y faire.

Le diagramme de séquence illustre les interactions entre les différents objets. Dans l'exemple ci-dessus, j'ai créé un diagramme pour deux cas d'utilisation très courants: la connexion/l'inscription et la création de personnage.

Deployment

Le processus de déploiement est un peu plus simple que d'habitude grâce au choix d'utiliser Docker dès le début du projet. Même si mes images et conteneurs de développement ne peuvent pas être utilisés exactement de la même manière, l'environnement et la configuration restent similaires, ce qui rend le processus plus facile et plus rapide.

1. La première étape du processus de déploiement consiste à choisir l'hébergeur, et j'ai déjà opté pour DigitalOcean.
2. Sur DigitalOcean, j'ai suivi l'interface utilisateur pour créer un projet et un droplet au sein de ce projet. Pour mon cas d'utilisation, j'ai opté pour un VPS moins puissant, car l'utilisation prévue du projet est modérée. J'ai également connecté ma clé publique SSH afin de pouvoir me connecter immédiatement au VPS depuis mon ordinateur portable. Concernant la localisation, DigitalOcean propose les États-Unis ou le Royaume-Uni. j'ai choisi le Royaume-Uni, car c'est de là que se connecteront la plupart des utilisateurs.
3. Je me connecte au VPS via SSH en utilisant la commande ``ssh root@votre_adresse_ip``. Cette commande utilise ma clé SSH pour l'authentification et me permet d'accéder immédiatement au serveur.
4. Je commence par mettre à jour les paquets sur le serveur à l'aide de la commande ``apt update && apt upgrade -y``

Création d'un nouvel utilisateur:

Pour des raisons de sécurité, j'ai créé un nouvel utilisateur pour le serveur et supprimé l'accès de l'utilisateur root pour le VPS.

1. J'ai créé le nouvel utilisateur avec ``adduser userecf`` et ajouté un mot de passe.
2. L'utilisateur a été ajouté au groupe sudo pour obtenir l'accès à la commande sudo pour les opérations nécessitant des permissions plus élevées à l'aide de la commande ``usermod -aG sudo userecf``.
3. J'ai copié la clé ssh de l'utilisateur root dans le nouvel utilisateur à l'aide de la commande rsync : ``rsync --archive --chown=userecf:userecf ~/.ssh /home/userecf``
4. Maintenant que le nouvel utilisateur est prêt à être utilisé, j'ai désactivé l'accès root et par mot de passe en modifiant le fichier `/etc/ssh/sshd_config`.

Configuration du pare-feu:

Avant de redémarrer le service ssh et de passer au nouvel utilisateur que j'ai créé, j'ai configuré le pare-feu pour autoriser l'accès au serveur uniquement via ssh, ou les ports par défaut pour http et https, qui sont 80 et 443, en utilisant les commandes suivantes:

1. `ufw allow OpenSSH`
2. `ufw allow 80/tcp`
3. `ufw allow 443/tcp`
4. `ufw enable`

Maintenant que les mesures de sécurité de base ont été mises en œuvre, j'ai utilisé ``exit`` et ``ssh userecf@votre_adresse_ip_serveur`` pour me reconnecter en utilisant le nouvel utilisateur que j'ai créé.

Installation de Docker:

Comme j'utilise Docker, je n'ai rien d'autre à installer que Git, Docker et Docker Compose.

Git peut être facilement installé en utilisant ``sudo apt install git -y``

Pour Docker, j'ai suivi un guide communautaire pour Ubuntu sur le site web Digital Ocean.

1. J'ai installé certains paquets prérequis: ``sudo apt install ca-certificates curl gnupg``
2. Ajout de GPG du dépôt Docker officiel au système ``sudo install -m 0755 -d /etc/apt/keyrings``
`curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`
`sudo chmod a+r /etc/apt/keyrings/docker.gpg``
3. Ajout du dépôt Docker aux sources APT : ``echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null``
4. ``sudo apt update`` pour appliquer les modifications
5. Enfin, installez Docker : ``sudo apt install docker-ce``
6. Je peux vérifier l'état du service en utilisant ``sudo systemctl status docker``
7. Dernière étape: ajouter l'utilisateur au groupe docker pour éviter d'utiliser sudo à chaque exécution de la commande docker. ``sudo usermod -aG docker userecf``

Installation de Docker Compose:

Comme nous avons déjà effectué une grande partie des étapes de configuration de Docker pour installer Docker, l'installation du plugin Docker Compose est beaucoup plus simple et nous pouvons le faire en utilisant simplement cette commande :

``sudo apt install docker-compose-plugin``

Configuration du projet:

1. Accédez au répertoire `/opt/` et clonez le projet depuis GitHub avec la commande ``git clone github_link``, puis placez-vous dans ce répertoire.
2. Copiez le fichier `.env` d'exemple et modifiez les valeurs pour votre environnement de production : ``sudo cp .env.example .env``. Il est crucial de modifier ``APP_DEBUG=false`` pour des raisons de sécurité, sans cela, les messages de débogage pourraient divulguer des informations sensibles aux utilisateurs.

J'ai également créé un dossier `prod` dans le répertoire Docker du projet Laravel Sail, contenant mon fichier `nginx.conf`, mon Dockerfile PHP et mon fichier `php.init`. J'ai ensuite créé un fichier `docker-compose` de production à la racine, qui lance plusieurs conteneurs et leur permet de communiquer entre eux.

3. Nous pouvons maintenant utiliser la commande `docker compose -f compose.prod.yaml up -d --build` pour construire et exécuter le conteneur.

4. Après cela, j'ai accédé au shell Docker de l'application pour terminer la configuration en utilisant `docker exec -it app sh`

Commandes PHP Artisan générales pour la configuration de production Laravel:

- `php artisan key:generate`
- `php artisan migrate`
- `php artisan db:seed --class=PermissionsSeeder`
- `php artisan db:seed --class=UserSeeder`
- `php artisan optimize`
- `php artisan queue:work --sleep=3 --tries=3 --timeout=90 --memory=256`

Finalement, le déploiement complet a été achevé par la mise en place du service certbot et d'une tâche cron pour automatiser le certificat SSL.

1. `sudo apt install certbot`
2. `sudo mkdir -p /etc/letsencrypt`
3. `sudo mkdir -p /var/www/certbot`
4. `sudo chmod -R 755 /var/www/certbot`
5. `sudo certbot certonly --webroot --webroot-path=/var/www/certbot --email your@email.com --agree-tos --no-eff-email -d ecf.smor.dev -d www.ecf.smor.dev`
6. `systemctl status certbot.timer`
7. `sudo crontab -e`
8. J'ai ajouté cette ligne au fichier. `0 3 * * * certbot renew --quiet && docker exec pixelverse_nginx nginx -s reload`

Une fois toutes ces étapes réalisées, j'ai sécurisé le serveur avec un nouvel utilisateur et un pare-feu, puis installé Docker et Docker Compose. J'ai créé mon image Docker et le conteneur correspondant, que j'ai utilisés pour configurer les applications Laravel et activer le worker de file d'attente. J'ai également configuré un certificat SSL avec Certbot et ajouté une tâche cron sur le serveur pour automatiser son renouvellement à expiration, afin de rediriger toutes les requêtes HTTP vers HTTPS pour des raisons de sécurité.

Le produit final est accessible sur le sous-domaine `ecf.smor.dev`, car j'y ai connecté un domaine existant que je possédais déjà et l'ai rattaché au VPS à l'aide d'enregistrements A pointant vers l'adresse IP du serveur.